

(Advanced Java Programming)

Python, NAST

2022 - Spring

1.a) Define multimedia. Explain each component in details.

eg

→ Multimedia means that computer information can be represented through audio, video and animation in addition to traditional media (i.e. text, graphics and images).

→ The components of multimedia are discussed below:

i) Text :

→ In text, characters are used to form words, phrases and paragraphs.

→ Text can be ~~variety~~ in a variety of fonts and sizes to ~~match~~ match the multimedia software's professional presentation.

→ Text formats : .docx, .pdf, .txt etc.

ii) Graphics:

→ Graphics is a digital representation of non-text information such as drawing, chart or paragraph.

→ Graphics are at the heart of any multimedia presentation.

→ Graphics formats : .jpeg, .png, .gif etc.

iii> Audio:

- Audio is any sound, whether it's music, conversation or something else.
- Audio is the most serious aspect of multimedia, delivering the joy of music, special effects and other forms of entertainment.
- Audio formats: .mp3, .WMA, .wave etc.

iv> Video:

- Video is the photographic images that appear to be in full motion.
- Video is ~~the~~ a moving image that is accompanied by sound, such as television picture.
- Video formats: .mp4, .3GP, .mkv ~~etc~~, .MOV etc.

v> Animation:

- Animation is a series of graphics that create an illusion of motion.
- Animation formats are as similar to video formats.

2. a) What is the role of hibernate in software development?
Write down - Hibernate Annotations with example.

→ The role of hibernate in software development is that hibernate reduces lines of code by maintaining object-table mapping itself and returns result to application in the form of Java objects.

The different hibernate annotations are discussed below:

i) @Entity :

→ This is used for declaring any POJO class as an entity for a database.

ii) @Table :

→ This is used to change table details.

iii) @Id :

→ This is used for declaring a primary key inside ~~any~~ POJO class.

iv) @GeneratedValue :

→ This is used to generate the values with reference to the internal sequence automatically.

v) @Column:

→ This is used to specify column mappings.

vi) @OrderBy:

→ This is used to sort data in required order.

vii) @Transient:

→ This tells the hibernate, not to add this particular column.

viii) @Lob :

→ This tells the hibernate, it is a large object and is not a simple object.

* For example:

@Entity

@Table (name = "company")

public class company implements Serializable {

 @Id

 @Column (name = "id")

 @GeneratedValue

 private int id;

 @OrderBy ("firstName asc")

 private Set contacts;

}

2.b) Define the use of SpringBoot.

507

⇒ Spring boot is an open-source Java framework used for programming standalone, production-grade Spring based applications with minimal effort.

(Advantages)

* Features of Spring Boot:

- i) Spring boot creates stand-alone Spring applications.
- ii) Spring boot embeds Tomcat, Jetty or Undertow directly.
- iii) It does not require XML configuration.
- iv) It minimizes writing multiple boilerplate codes, XML configuration and annotations.
- v) It offers the number of plug-ins.
- vi) It increases productivity and reduces development time.

The different Spring Boot Annotations with example are discussed below:

i) > @SpringBootApplication:

- This is used to mark the main class of a Spring Boot application.
- It combines @Configuration, @EnableAutoConfiguration and @ComponentScan.
- For example:

```
@SpringBootApplication
public class SpringBoot {
    public static void main (String [] args) {
        SpringApplication.run (SpringBoot.class, args);
    }
}
```

ii) > @Autowired:

- This is applied on fields, setter methods and constructors.
- For example;

```
public class Customer {
    @Autowired
    private Person person;
    private int type;
}
```


iii> @Required:

→ This is applied on bean setter methods.

iv> @Qualifier:

→ This is used along with @Autowired annotation.

v> @Configuration:

→ This is used on classes with define beans.

vi> @Bean:

→ This is used at the method level.

vii> @Lazy:

→ This is used on component classes.

viii> @Service:

→ This is used on a class.

3.a) Define Localization and Internationalization with example.

89

* Localization:

- Localization is the process of adapting software for a specific language and region by adding locale-specific components and translating text.
- It is abbreviated as ~~L10N~~ L10N because there are 10 letters between L and N.

* Internationalization:

- Internationalization is the processing of designing an application for various languages and regions without engineering changes.
- It is abbreviated as I18N because there are 18 letters between I and N.

* Before Internationalization:

```
public class NotI18N {  
    static public void main (String [] args) {  
        System.out.println ("Hello");  
        System.out.println ("How are you?");  
        System.out.println ("Good Bye");  
    }  
}
```


After Internationalization:

```
import java.util.*;
```

```
public class I18N {
```

```
    static public void main (String [] args) {
```

```
        String language;
```

```
        String country;
```

```
        if (args.length != 2) {
```

```
            language = new String ("en");
```

```
            country = new String ("US");
```

```
        } else {
```

```
            language = new String (args [0]);
```

```
            country = new String (args [1]);
```

```
        }
```

```
        Locale currentLocale
```

```
        ResourceBundle messages;
```

```
        currentLocale = new Locale (language, country);
```

```
        messages = ResourceBundle.getBundle ("Messages  
Bundle", currentLocale);
```

```
        System.out.println (messages.getString ("greeting"));
```

```
        System.out.println (messages.getString ("inquiry"));
```

```
        System.out.println (messages.getString ("farewell"));
```

```
    }
```

```
}
```


4.6> What are the rules to create POJO class? Write down simple POJO class having at least 4 properties.

89

→ POJO stands for Plain Old Java object.

→ The rules to create POJO class are as follows:

i> class Declaration

ii> Default Constructor

~~iii> ~~Private~~ Private Getters and Setters~~

iii> Private Properties

iv> Public methods

v> Public Getters and setters.

* Simple POJO class:

~~Public~~

```
public class Student {
```

```
    private Long id;
```

```
    private name String name;
```

```
    private String address;
```

```
    private String email;
```

```
    public Student () {
```

```
        id = 1001;
```

```
        name = "Prabin Chaudhary";
```

```
        address = "Dhangadhi";
```

```
        email = "prabin@gmail.com";
```

```
    }
```



```
public Long getId() {  
    return id;  
}
```

```
public void setId (Long id) {  
    this.id = id;  
}
```

```
public String getName () {  
    return name;  
}
```

```
public void setName (String name) {  
    this.name = name;  
}
```

```
public String address getAddress () {  
    return address;  
}
```

```
public void setAddress () {  
    this.address = address;  
}
```



```
public String getEmail() {  
    return email;  
}
```

```
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}
```

```
}
```

```
}
```


Define

5. a) EJB. Explain the different types used to create EJB Applications.

sof

- EJB stands for Enterprise JavaBeans.
- EJB is a server-side software component that encapsulates business logic of an application.

→ The different types of EJB are listed below:

!

i) Session Beans:

- Session Beans are used to encapsulate business logic that can be invoked by a client.

ii) Entity Beans:

- Entity Beans are used to represent persistent data stored in a database.

iii) Message-Driven Beans (MDB):

- Message-Driven Beans are used to process messages asynchronously.

* Architecture of EJB :

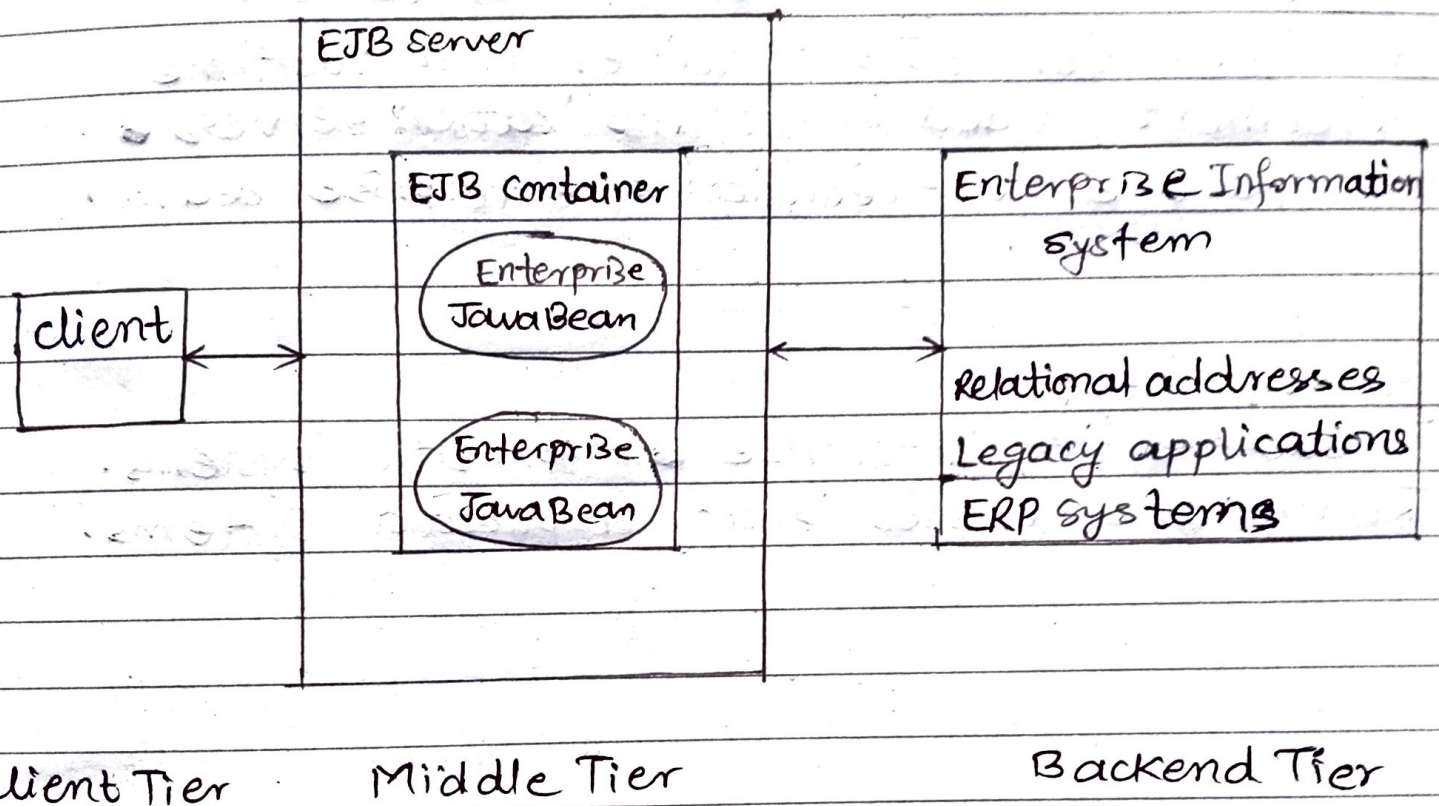


Fig. EJB Architecture :

1> client Tier:

- ~~EJB client~~ In client Tier, EJB client represents the end-users who interact with the application.
- EJB client may be a web browser, a mobile device or a standalone application.

2) Middle Tier:

- In middle tier, EJB server provides the execution environment for EJB containers.
- In middle tier, EJB container is a runtime environment provided by the application server that manages the execution of enterprise beans.

3) Backend Tier:

- In Backend tier, Enterprise information systems.
- Relational addresses legacy application systems.

JSP chapter-3

6.a) Define JSP. Explain the life cycle of JSP along with compilation process in details.

SP

- JSP stands for Java Server Pages.
- JSP is a server side technology that is used to create dynamic, platform-independent web-application.
- The life cycle of JSP is shown below:

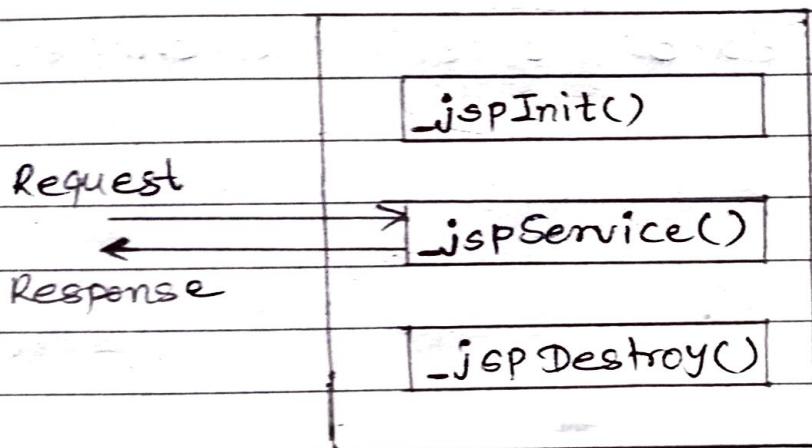


Fig. Life cycle of JSP

i) `_jspInit()` :

- This method is declared on the JSP page.
- This method is called only once in the complete life cycle of JSP.

ii) `jspService()` :

- This method is invoked by the JSP container for each client request.
- This method passes the request and response objects.

iii) `jspDestroy()` :

- This method is used for shutting down the application / container.
- This method is also called only once in the complete lifecycle of JSP.

Compilation Process of JSP:

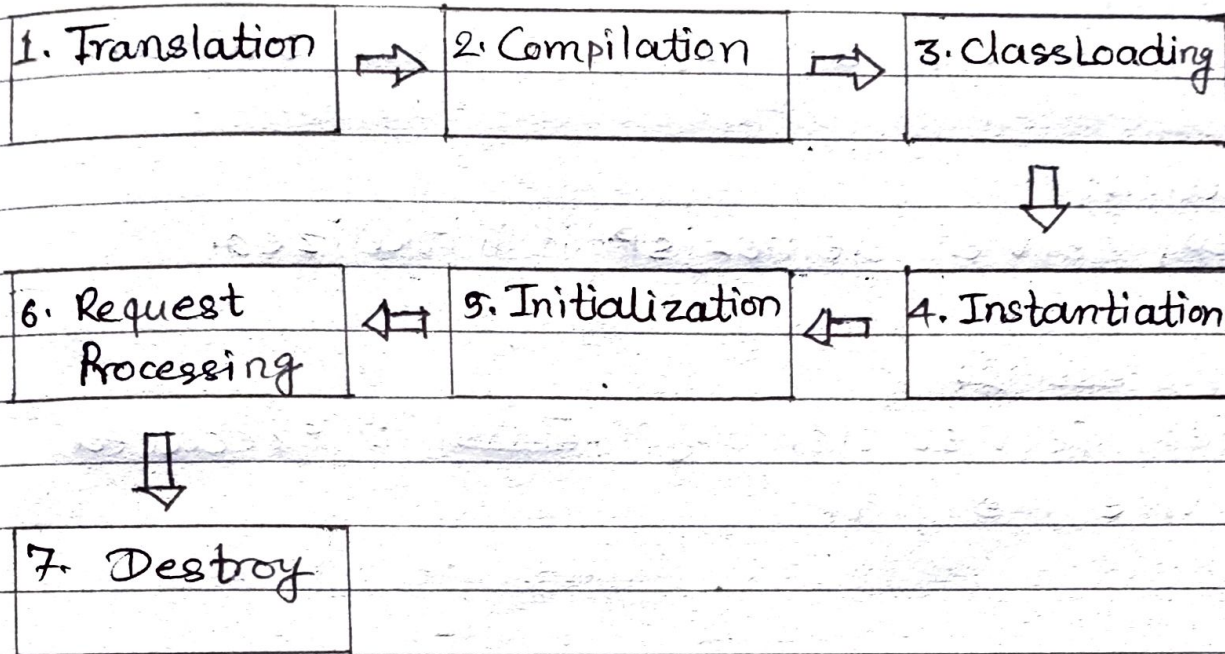


Fig. Compilation Process of JSP

i) Translation:

→ In this step, demo.jsp is translated into servlet.

ii) Compilation:

→ In this step, servlet demo-jsp.java is created.

iii) ClassLoading:

→ In this step, demo-jsp.java is loaded into demo-jsp.class.

iv> Instantiation:

→ In this step, servlet demo.jsp is instantiated.

v> Initialization:

→ In this step, servlet demo.jsp is initialized.

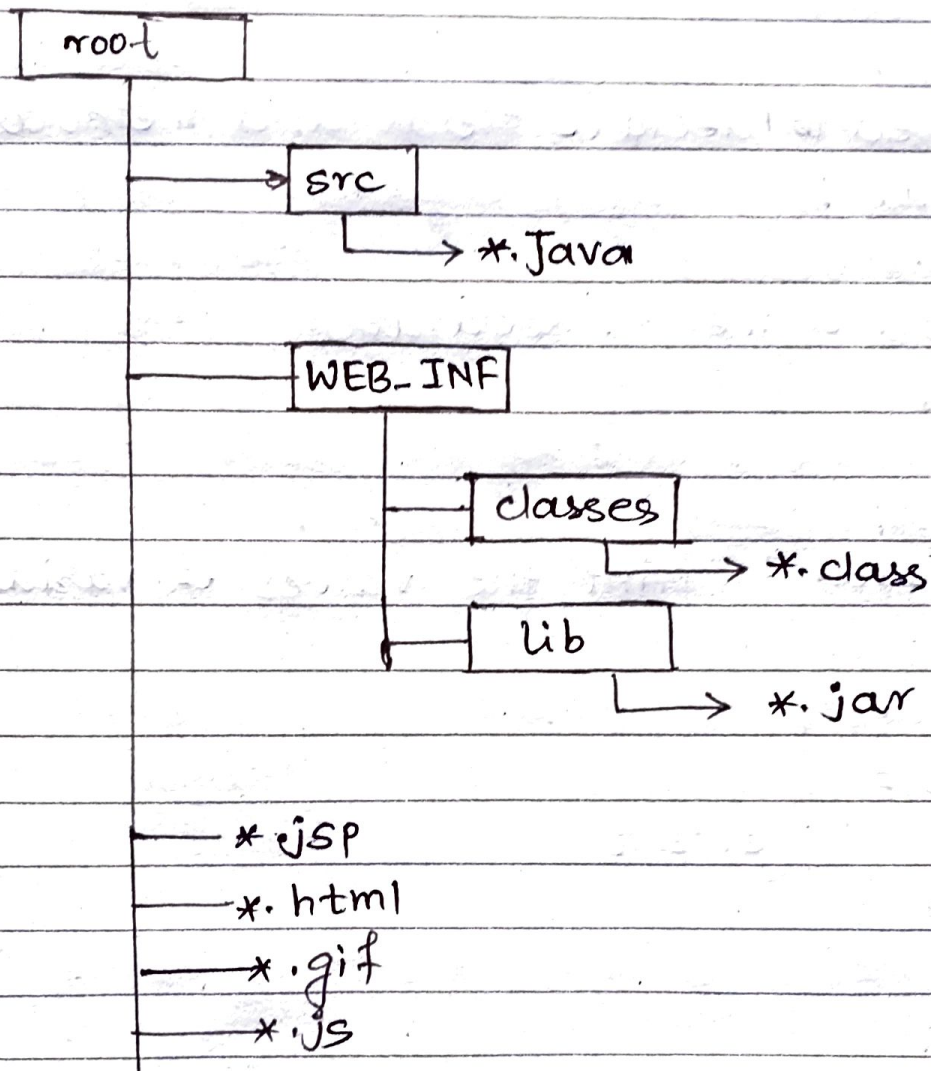
vi> Request Processing:

→ In this step, Servlet demo.jsp ~~Java~~ is executed using service method.

vii> Destroy:

→ In this step, servlet demo.jsp is destroyed.

(Folder) #JSP Directory Structure



JSP Syntax:

i) Declaration:

→ This is used to declare fields and methods.

→ Its syntax:

<% !

field or method declaration

%>

ii) Expression:

→ This is used to print the values of variable or method.

→ Its syntax:

<% =

statement

%>

iii) Scriptlet:

→ This is used to execute java source code.

→ Its syntax:

<%

java source code

%>

iv) Directives:

→ This is used to define attributes that apply to an entire JSP page.

→ It's syntax:

`<%@ page directive attribute = "value" %>`

JSP implicit Objects:

→ Implicit objects are the set of Java Objects that the JSP container makes available to developer on each page.

→ There are 9 implicit objects on JSP which are discussed below:-

i> request:

→ request is the object of `HttpServletRequest` class associated with the request.

ii> response:

→ response is the object of `HttpServletResponse` class associated with the response to the client.

iii> config:

→ config is the object of `ServletConfig` class associated with the page.

iv> application:

→ application is the object of `ServletContext` class associated with the application context.

v> session:

→ session is the object of `HttpSession` class associated with the request.

vi> page context:

→ page context is the object of `PageContext` class that is used to find, get or remove an attribute.

vii> Page object:

→ Page object is used to refer to the current translated servlet class.

viii> exception:

→ exception object represents all errors and exceptions that is accessed by the respective jsp.

ix> out:

→ out is the `PrintWriter` object used for displaying the contents to the client.

#Tomcat:

- Tomcat is an open-source web server and servlet.
- Tomcat was developed by Apache Software Foundation.
- It is ~~is~~ widely used for hosting Java-based applications on the web.
- Tomcat is built on Java technologies and implements the Java Servlet and JSP specifications.
- Tomcat acts as a bridge between web server and web-based applications.

use of `<jsp:include>` and `<jsp:forward>` :

i) `<jsp:include>`:

→ It is used to include the content of another resource at the time the JSP is requested.

→ For example:

```
<jsp:include page = "header.jsp" />
```

```
<P> Main contents </P>
```

```
<jsp:include page = "footer.jsp" />
```

ii) `<jsp:forward>`:

→ It is used to forward the request from one JSP to another resource.

→ For example:

```
<%
```

```
    boolean condition = true;
```

```
    if (condition) {
```

```
        <%
```

```
            <jsp:forward page = "success.jsp" />
```

```
        %>
```

```
    } else {
```

```
        <%
```

```
            <jsp:forward page = "error.jsp" />
```

```
        %>
```

```
    }
```

```
%>
```

GET Method vs POST Method:

GET Method	POST Method
i> GET method is used to retrieve information from the server.	POST method is used to send data to server to create/update a resource.
ii> GET method is displayed in the URL.	POST method is not displayed in the URL.
iii> GET method is Idempotent.	POST method is non-Idempotent.
iv> GET method can be cached.	POST method cannot be cached.
v> GET method is ideal for searching and retrieving data.	POST method is ideal for transactions and updating data.

JSP Architecture:

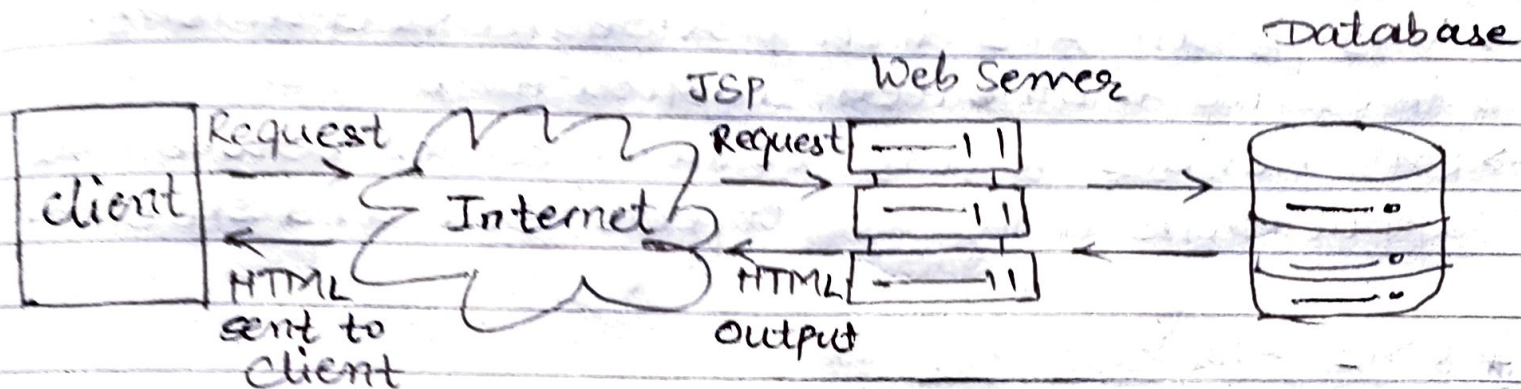


Fig. JSP Architecture

- JSP architecture is a 3-tier architecture that separates a web application's presentation, logic and data layers.
- Presentation layer or client side is responsible for displaying the user interface and handling user interaction.
- The logic layer or server side is responsible for processing user requests and handling business logic.
- The data layer ~~or database~~ is responsible for storing and retrieving data from a database.

RMI and its architecture:

- RMI stands for Remote Method Invocation.
- RMI is an API that provides a mechanism to create distributed application.
- RMI provides remote communication between the applications using two objects stub and skeleton.

RMI Architecture:

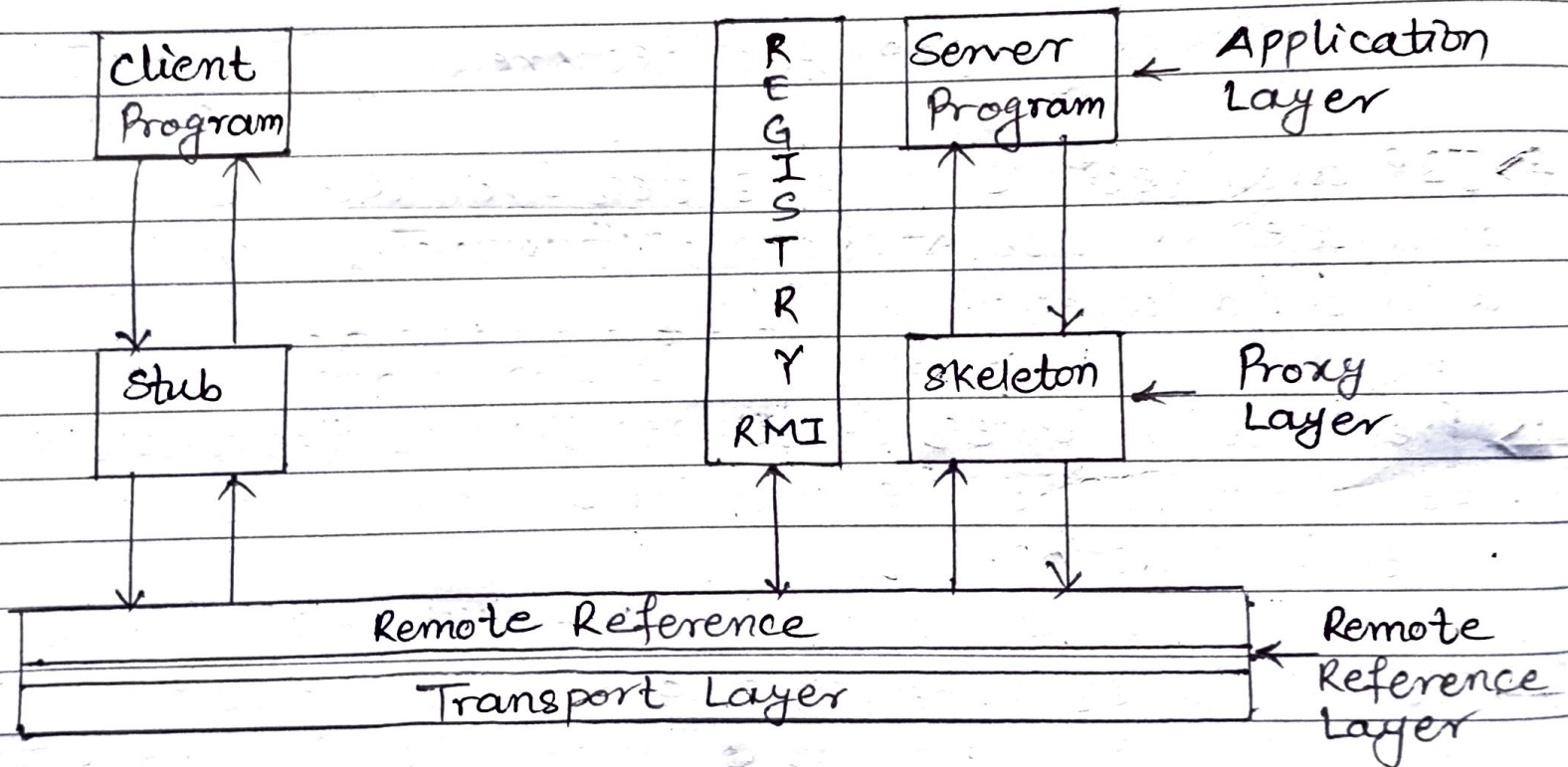


Fig. RMI Architecture

- The RMI architecture is very simple involving a client program, a server program, a stub and a skeleton.

i> Application layer:

→ Application layer is nothing but the actual systems (client and server) involved in communication.

ii> Proxy Layer:

→ The proxy layer consists of proxies (i.e. stub and skeleton) for client and server.

iii> Remote Reference Layer:

→ Remote Reference Layer is responsible for object communication and transfer of objects between client and server.

iv> Transport layer:

→ Transport layer is responsible for actual setting up connections and handling the transport of data from one machine to another.

Hibernate Architecture:

- Hibernate is a framework used to develop persistence logic which is independent of Database software.
- The architecture of hibernate is shown below:

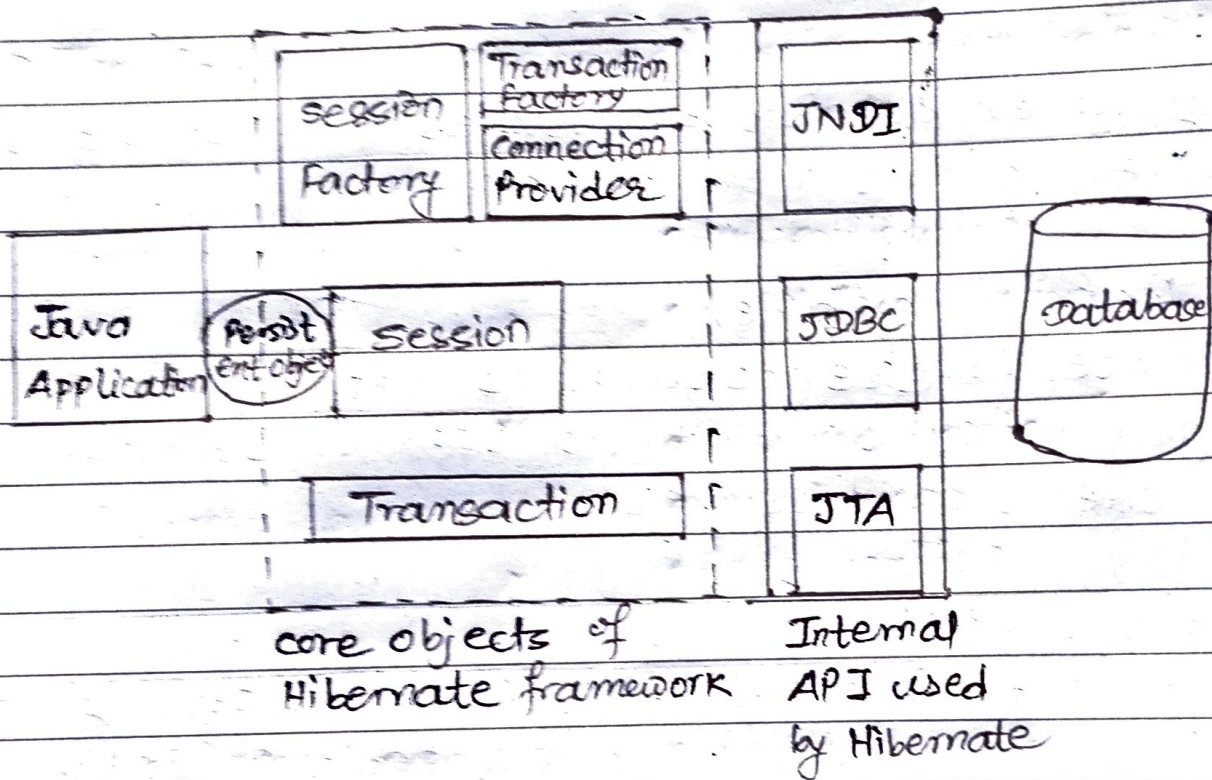


Fig. Hibernate Architecture

→ The elements of hibernate architecture are shown below:-

1) SessionFactory:

→ SessionFactory is an interface which is present in org.hibernate package and it is used to create SessionObject.

ii> Session;

→ Session is an interface which is present in org.hibernate package. Session object is created based upon session SessionFactory object i.e. factory.

iii> Transaction;

→ The transaction object specifies the atomic unit of work.

iv> Connection Provider;

→ Connection Provider is a factory of JDBC connections.

v> TransactionFactory;

→ TransactionFactory is the factory of Transaction.

Spring Boot architecture:

→ Spring Boot is an open source Java framework used for programming standalone, production grade spring based applications with minimal support.

→ The Spring Boot architecture is shown below:

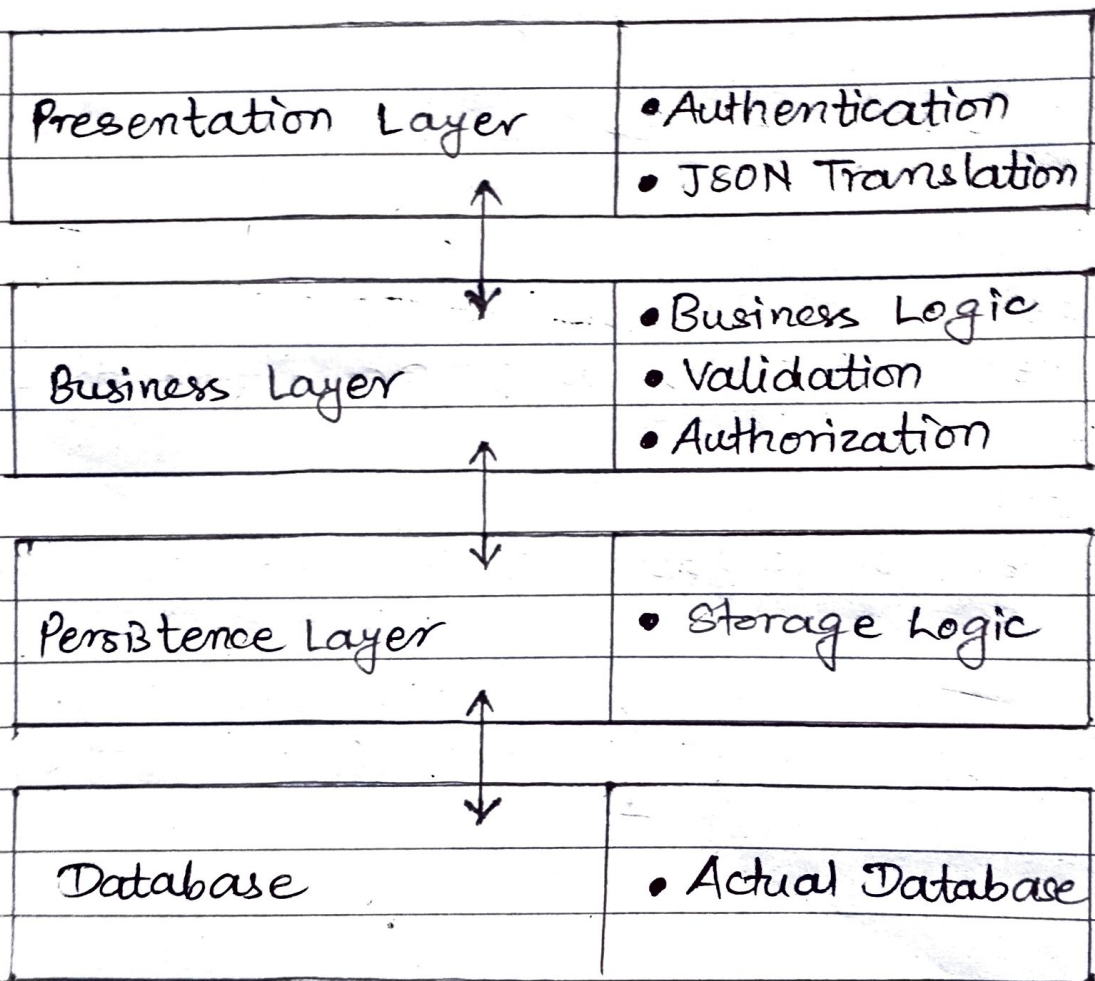


Fig. Spring Boot Architecture

i> Presentation Layer:

→ The presentation layer handles the HTTP requests, translates the JSON parameter to object and authenticates the request and transfers it to the business layer.

ii> Business Layer:

→ The business layer handles all the business logic and also performs validation and authorization.

iii> Persistence Layer:

→ The persistence layer contains all the storage logic and translates business objects from and to database rows.

iv> Database layer:

→ The database layer performs CRUD operations i.e. create, retrieve, update and delete.